Aula 01 — Lógica de Programação

Como transformar problemas em passos, algoritmos e código

Dieimes Nunes

Boas-vindas

Curso: Lógica de Programação
Carga horária: 60 horas
Linguagem principal: Python

Nesta aula, vamos começar com uma ideia importante:

> Programar não começa no computador.
> Programar começa quando entendemos um problema e organizamos uma solução.

Quem sou eu



Dieimes Nunes

Instrutor de Informática e Programação

Atuo com ensino de tecnologia, programação, desenvolvimento de sistemas e formação profissional.

Minha caminhada na área envolve:

- Análise e Desenvolvimento de Sistemas;
- Gestão e Governança da Tecnologia da Informação;
- Docência no Ensino Técnico;
- estudos na área de Ciência da

Minha forma de trabalhar com vocês

Aqui, ninguém precisa chegar sabendo programar.

A proposta do curso é aprender aos poucos:

1. entender o problema;
2. pensar em passos;
3. testar a ideia no papel;
4. transformar em código;
5. errar, corrigir e melhorar.

Erro em programação não é fracasso.

Como será nosso curso

Durante o curso, vamos estudar:

- lógica de programação;
- algoritmos;
- variáveis;
- entrada e saída de dados;
- operadores;
- estruturas de decisão;
- estruturas de repetição;
- vetores, matrizes e registros;
- funções;
- testes de mesa e depuração;
- noções de programação orientada a objetos.

Antes do código: uma pergunta

Imagine que alguém diga:

> Faça um programa para calcular a média de um aluno.

Antes de programar, precisamos perguntar:

- Quantas notas serão usadas?
- A média será simples ou ponderada?
- O aluno será aprovado com qual nota?
- O programa deve mostrar apenas a média ou também a situação?

O que é lógica?

Lógica é uma forma de organizar o pensamento.

Ela ajuda a responder perguntas como:

- O que precisa ser feito primeiro?
- Quais informações eu tenho?
- Quais informações estão faltando?
- Qual resultado eu quero alcançar?
- Qual caminho faz mais sentido?

Na programação, lógica é a base para criar soluções.

Exemplo simples de lógica

Situação: usar um computador.

Uma sequência possível seria:

1. Ligar o computador.
2. Aguardar o sistema iniciar.
3. Informar usuário e senha.
4. Abrir o programa desejado.
5. Utilizar o programa.
6. Encerrar o programa.
7. Desligar o computador.

A ordem importa.

Quando a ordem está errada

Veja esta sequência:

1. Digitar a senha.
2. Abrir o programa.
3. Ligar o computador.
4. Aguardar o sistema iniciar.

Discussão rápida:

> O que está errado nessa sequência?

Atividade 1 — Organizando uma sequência

No caderno, organize corretamente os passos abaixo.

Situação: acessar uma conta de e-mail.

```
Abrir a caixa de entrada.
Digitar a senha.
Abrir o navegador.
Ligar o computador.
Acessar o site do e-mail.
Digitar o usuário.
Clicar em entrar.
Aguardar o sistema iniciar.
```

Depois, compare com um colega.

Discussão da Atividade 1

Uma possível ordem seria:

1. Ligar o computador.
2. Aguardar o sistema iniciar.
3. Abrir o navegador.
4. Acessar o site do e-mail.
5. Digitar o usuário.
6. Digitar a senha.
7. Clicar em entrar.
8. Abrir a caixa de entrada.

Pergunta:

> Algum passo ainda poderia ser detalhado melhor?

O que é programação?

Programação é o processo de criar instruções para que o computador execute uma tarefa.

Mas o computador não entende intenção.

Ele entende instruções.

Essas instruções precisam ser:

O computador é obediente, mas não adivinha

Imagine a instrução:

```
Some os números.
```

Isso parece simples, mas está incompleto.

Perguntas importantes:

- Quais números?
- Quem vai informar esses números?
- O resultado será mostrado onde?
- O programa precisa guardar o resultado?

Na programação, instruções vagas geram problemas.

Atividade 2 — Melhorando instruções vagas

Reescreva as instruções abaixo deixando-as mais claras.

Instrução A

```
Calcule a média.
```

Instrução B

```
Cadastre o aluno.
```

Instrução C

```
Mostre o resultado.
```

Tente explicar **quais dados entram**, **o que será feito** e **o que deve aparecer no final**.

Exemplo de melhoria

Instrução vaga:

```
Calcule a média.
```

Instrução mais clara:

```
Receba a primeira nota do aluno.
Receba a segunda nota do aluno.
Some as duas notas.
Divida o resultado por 2.
Mostre a média final na tela.
```

Agora a tarefa está mais organizada.

O que é um algoritmo?

Um **algoritmo** é uma sequência organizada e finita de passos para resolver um problema ou realizar uma tarefa.

Exemplos de algoritmos no cotidiano:

- receita de bolo;
- manual de montagem;
- rota no GPS;
- saque no caixa eletrônico;
- processo de login;
- cálculo de média;
- cadastro de cliente;
- compra em uma loja.

Características de um algoritmo

Um bom algoritmo precisa ter:

- um problema a resolver;
- início;
- passos em ordem;
- instruções claras;
- fim;
- resultado esperado.

Um algoritmo não pode depender de adivinhação.

Se uma pessoa não consegue seguir os passos, provavelmente o computador também não conseguirá.

Exemplo de algoritmo cotidiano

Algoritmo: escovar os dentes

1. Pegar a escova.
2. Colocar creme dental.
3. Molhar a escova.
4. Escovar os dentes.
5. Enxaguar a boca.
6. Lavar a escova.
7. Guardar a escova.
8. Encerrar.

Parece simples, mas já existe uma sequência lógica.

Atividade 3 — Algoritmo do cotidiano

Escolha uma tarefa e escreva um algoritmo com pelo menos **8 passos**.

Opções:

- ligar o computador e acessar a internet;
- enviar uma mensagem pelo celular;
- preparar um café;
- fazer uma compra no mercado;
- acessar o e-mail;
- realizar um saque no caixa eletrônico.

Depois, troque com um colega e peça para ele verificar se os passos estão claros.

Como revisar o algoritmo do colega

Observe:

- A ordem dos passos faz sentido?
- Algum passo importante ficou faltando?
- Existe alguma instrução confusa?
- O algoritmo tem começo e fim?
- Uma pessoa conseguiria seguir esses passos sem perguntar nada?

Essa revisão é parecida com o que fazemos ao testar um programa.

Entrada, processamento e saída

Muitos programas seguem esta estrutura:

```
ENTRADA → PROCESSAMENTO → SAÍDA
```

Ou seja:

- o programa recebe dados;
- faz algum processamento;
- mostra ou grava um resultado.

Esse modelo ajuda a entender quase qualquer programa simples.

Exemplo: média de notas

Problema:

> Receber duas notas e calcular a média.

Entrada:

- nota 1;
- nota 2.

Processamento:

```
media = (nota1 + nota2) / 2
```

Saída:

- média final.

Exemplo: compra em uma loja

Problema:

> Calcular o valor total de uma compra.

Entrada:

- nome do produto;
- preço;
- quantidade.

Processamento:

```
total = preço * quantidade
```

Saída:

- valor total da compra.

Atividade 4 — Entrada, processamento e saída

Para cada situação, identifique a **entrada**, o **processamento** e a **saída**.

Situação A

Receber dois números e mostrar a soma.

Situação B

Receber três notas e calcular a média.

Situação C

Receber o preço de um produto e o percentual de desconto. Depois, calcular o valor final.

Situação D

Receber a distância de uma viagem e o consumo médio do veículo. Depois, calcular a quantidade estimada de combustível.

Modelo para responder

Use este modelo no caderno:

```
Situação: ____________________________

Entrada:
-
-

Processamento:
-

Saída:
-
```

O importante é separar o que o programa **recebe**, o que ele **faz** e o que ele **entrega**.

Pseudocódigo

Pseudocódigo é uma forma de escrever um algoritmo usando uma linguagem parecida com a linguagem humana.

Ele não precisa ser executado pelo computador.

Ele serve para organizar a ideia antes do código.

Pense nele como um rascunho do programa.

Exemplo de pseudocódigo

```
INÍCIO

    ESCREVA "Digite a primeira nota:"
    LEIA nota1

    ESCREVA "Digite a segunda nota:"
    LEIA nota2

    media ← (nota1 + nota2) / 2

    ESCREVA "A média é:", media

FIM
```

Entendendo o pseudocódigo

No exemplo anterior, temos:

- `INÍCIO`: marca onde o algoritmo começa;
- `ESCREVA`: mostra uma mensagem;
- `LEIA`: recebe uma informação;
- `media`: guarda o resultado do cálculo;
- `FIM`: marca onde o algoritmo termina.

Ainda não estamos preocupados com a sintaxe perfeita.

Estamos preocupados com o raciocínio.

Atividade 5 — Criando um pseudocódigo

Crie um pseudocódigo para uma das situações abaixo.

Opção 1

Receber o nome de uma pessoa e mostrar uma mensagem de boas-vindas.

Opção 2

Receber o nome de um produto, o preço e a quantidade. Depois, mostrar os dados da compra.

Opção 3

Primeiro contato com Python

Python é uma linguagem de programação muito usada em diferentes áreas, como:

- automação de tarefas;
- análise de dados;
- inteligência artificial;
- desenvolvimento web;
- ensino de programação;
- criação de scripts.

Primeiro programa

```
print("Olá, mundo!")
```

O comando `print()` mostra uma informação na tela.

Neste exemplo, o texto `Olá, mundo!` aparece para o usuário.

Atenção:

- textos ficam entre aspas;
- os parênteses fazem parte do comando;
- cada detalhe da escrita importa.

Testando outros textos

```
print("Bem-vindo ao curso de Lógica de Programação!")
print("Hoje é nosso primeiro contato com Python.")
print("Vamos aprender a resolver problemas passo a passo.")
```

Cada `print()` mostra uma linha na tela.

O programa executa de cima para baixo.

Comentários em Python

Comentários são anotações dentro do código.

Eles ajudam a explicar o que está acontecendo.

```
# Este é meu primeiro programa em Python
print("Olá, turma!")
```

O Python ignora a linha que começa com #.

Comentários são úteis para estudar e revisar o código depois.

Atividade 6 — Primeiras mensagens em Python

Crie um arquivo chamado:

`aula01_mensagens.py`

Depois, escreva um programa que mostre:

- uma mensagem de boas-vindas;
- seu nome;
- o nome do curso;
- uma frase dizendo o que você espera aprender.

Use pelo menos **4 comandos** `print()`.

Variável: primeira ideia

Uma variável é um espaço usado para guardar uma informação.

Exemplo:

```
nome = "Ana"
idade = 16
```

Neste momento, pense assim:

> A variável é uma caixinha com um nome, onde guardamos um valor.

O nome da variável ajuda a entender o que está guardado nela.

O que acontece neste código?

```
nome = "Ana"
curso = "Lógica de Programação"

print(nome)
print(curso)
```

O programa:

1. guarda o texto `Ana` na variável `nome`;
2. guarda o texto `Lógica de Programação` na variável `curso`;
3. mostra o conteúdo da variável `nome`;
4. mostra o conteúdo da variável `curso`.

Entrada de dados com input()

O comando `input()` permite receber uma informação digitada pelo usuário.

```
nome = input("Digite seu nome: ")

print("Olá!")
print(nome)
```

Aqui, o usuário digita uma informação, e essa informação fica guardada na variável `nome`.

Melhorando a mensagem

```
nome = input("Digite seu nome: ")

print(f"Olá, {nome}! Seja bem-vindo ao curso.")
```

O `f` antes das aspas permite colocar variáveis dentro do texto.

Esse recurso é chamado de **f-string**.

Ele deixa a saída do programa mais organizada.

Exemplo com mais informações

```
nome = input("Digite seu nome: ")
cidade = input("Digite sua cidade: ")
curso = input("Digite o nome do curso: ")

print()
print("----- DADOS INFORMADOS -----")
print(f"Nome: {nome}")
print(f"Cidade: {cidade}")
print(f"Curso: {curso}")
```

O `print()` vazio pula uma linha.

Isso ajuda a organizar a saída na tela.

Atividade 7 — Programa de apresentação pessoal

Crie um programa em Python que solicite:

- nome;
- idade;
- cidade;
- curso;
- área da tecnologia que mais chama sua atenção.

Depois, mostre os dados organizados na tela.

Nome sugerido para o arquivo:

apresentacao.py

Modelo da Atividade 7

```
nome = input("Digite seu nome: ")
idade = input("Digite sua idade: ")
cidade = input("Digite sua cidade: ")
curso = input("Digite seu curso: ")
interesse = input("Qual área da tecnologia mais chama sua atenção? ")

print()
print("----- APRESENTAÇÃO -----")
print(f"Nome: {nome}")
print(f"Idade: {idade}")
print(f"Cidade: {cidade}")
print(f"Curso: {curso}")
print(f"Área de interesse: {interesse}")
```

Atividade 8 — Programa de cadastro simples

Crie um programa para cadastrar um produto.

O programa deve solicitar:

- nome do produto;
- categoria;
- preço;
- quantidade em estoque.

Depois, deve mostrar um resumo dos dados cadastrados.

Nesta aula, ainda não precisamos calcular o total do estoque.

Modelo da Atividade 8

```
produto = input("Digite o nome do produto: ")
categoria = input("Digite a categoria: ")
preco = input("Digite o preço: ")
quantidade = input("Digite a quantidade em estoque: ")

print()
print("----- PRODUTO CADASTRADO -----")
print(f"Produto: {produto}")
print(f"Categoria: {categoria}")
print(f"Preço: R$ {preco}")
print(f"Quantidade: {quantidade}")
```

Desafio da aula

Observe este programa:

```
produto = input("Produto: ")
preco = input("Preço: ")
quantidade = input("Quantidade: ")

print(f"Produto: {produto}")
print(f"Preço: {preco}")
print(f"Quantidade: {quantidade}")
```

Pergunta:

> Como poderíamos fazer o Python calcular o valor total da compra?

Guarde essa pergunta para a próxima aula.

Onde estamos no curso

Esta primeira aula abre a base do curso.

A sequência planejada é:

1. Entender problemas e criar algoritmos
2. Guardar informações com variáveis
3. Fazer cálculos e expressões
4. Tomar decisões com condições
5. Repetir tarefas com laços
6. Trabalhar com listas, matrizes, funções e arquivos

Hoje ficamos no primeiro passo e fizemos um contato inicial com Python.

Fechamento

Hoje vimos:

- lógica;
- programação;
- algoritmos;
- sequência lógica;
- entrada, processamento e saída;
- pseudocódigo;
- `print()`;
- `input()`;
- variáveis em uma primeira ideia.

Na próxima aula, vamos aprender como fazer cálculos com Python.

Perguntas finais para o caderno

1. O que é um algoritmo?
2. Por que a ordem dos passos é importante?
3. Qual é a diferença entre entrada, processamento e saída?
4. Para que serve o comando `print()`?
5. Para que serve o comando `input()`?
6. O que é uma variável?